

FLAKY-TEST RESEARCH ARTIFACTS · EVALUATION RUN

Three artifacts, one subject program

FlakyLens · FlakeSync · TSVD4J, run end-to-end against cricket-tracker

797 tests · 46 labelled flaky · 6 categories · every result scored against a hand-written ground truth

August 2026

cricket-tracker: a program built to be flaky

A working ball-by-ball cricket scoring service — 54 classes, 6,689 lines, Java 11 / Maven / JUnit 4, no external runtime dependencies — with flaky tests deliberately planted where such bugs naturally occur.

797

tests in the suite

46

labelled flaky (5.8%)

13

controls that must never fail

6

flakiness categories

4

Maven modules

Package	Category	n	What it exercises
com.cricket.live / .api	Async wait	11	sleep instead of awaitIdle() on the ingest pipeline
com.cricket.adversarial	Async wait (hidden)	8	every wait hidden behind a domain helper — day.drinks()
com.cricket.concurrent	Concurrency	7	unsynchronised shared state under a worker pool
com.cricket.od	Order dependency	12	9 victims / 3 brittles across 5 static state channels
com.cricket.time	Time	3	wall-clock budget assertions
com.cricket.uc	Unordered collections	4	iteration order of thread-spread accumulation

artifacts/flaky-labels.json is the ground truth — **every flaky test with its category, measured failure rate, and what each artifact is expected to report. All scoring below is against it.**

One laptop, no GPU, mixed shells

Component	What we ran on
CPU	AMD Ryzen 5 7520U — 4 cores / 8 threads
Memory	14 GB
GPU	AMD Radeon 610M — no NVIDIA, no CUDA
OS	Windows 11 Home, 10.0.26200
JDK	Temurin 11.0.32+9
Maven	3.9.16
Docker	27.5.1, WSL2 engine (FlakyLens only)
Python	3.13.4 on host; 3.8.5 + torch 2.3.1+cpu in the container

What the hardware cost us

No CUDA. FlakyLens hardcodes `torch.device("cuda")` — patched to auto-detect, then validated by re-running the paper's own fold.

4 cores. TSVD4J injects a 100 ms delay at every tracked access; the full run took 7 h 24 m.

Windows host. Artifact scripts are bash and assume POSIX; CRLF line endings broke both `patch(1)` and the container's shell scripts.

Everything below ran on this one machine. **Nothing was moved to a cluster or a GPU box — the ports and workarounds exist because of the row above.**

How we ran it

Fine-tuned CodeBERT that reads one test method and predicts its flakiness category: async wait, concurrency, time, unordered collections, order dependency, or non-flaky.

Corpus, container, inference

```
python tools/extract_tests.py --out artifacts/tests.json
python tools/to_flakylens_csv.py    # -> test_set_1.csv
docker build -t flakylens:latest .
docker run ... bash /app/run-cricket-tracker.sh
```

Inference only. fold-1 checkpoint, model.eval() under no_grad. 19 min for 797 tests on CPU.

Two runs. run A = the corpus as our converter emits it; run A couldn't detect any non-flaky tests. So, we had run B = the same corpus with one newline appended per test, nothing else changed. Result improved significantly.

Control first. the paper's own test_set_1.csv through the same patched pipeline, to prove the port did not change the model.

Nine changes to get it running here

Change	Why
utils.py — device auto-detect	no NVIDIA GPU
torch.load map_location	GPU-saved checkpoints on CPU
Dockerfile — conda-forge, CPU torch	conda ToS gate; 2.5 GB of CUDA wheels
.dockerignore, LF line endings	2 GB build context; bash rejects \r
attribution guard in Testing_*.py	per-test rows were silently dropped
to_flakylens_csv / analyze_flakylens	schemas differ; output has no test names

None of them alters a prediction — **verified twice: the device and checkpoint edits by the control run, the attribution guard by a byte-identical classification report before and after.**

Results

0.033

run A accuracy — as written

0.883

run B accuracy — + one newline

714 / 797

predictions that moved

19 min

per run, 797 tests on CPU

Tests classified correctly, by category

Category	n	Run A	Run B
async wait	20	12	12
concurrency	7	3	3
time	3	2	2
unordered collections	4	0	0
order dependency	12	9	0
non-flaky	751	0	687

Run A predicted non-flaky exactly zero times **out of 797 — 541 of its non-flaky tests were called order-dependent instead. Run B recovers 687 of 751, and loses all 12 real OD tests.**

The control — the paper's fold 1, our pipeline

Class	P	R	F1	n
0 async wait	0.50	0.73	0.59	11
1 concurrency	0.23	0.50	0.32	6
2 time	0.53	1.00	0.70	8
3 unordered coll.	0.64	0.78	0.70	9
4 order dependency	1.00	0.32	0.49	31
5 non-flaky	1.00	1.00	1.00	2164

Matches the artifact's published numbers on all six classes, **checked programmatically. The port is faithful — which means it reproduces the model's flaws faithfully too.**

Observations

A formatting label leak in the training data

Feature of a training row	Flaky	Non-flaky
ends with a newline	0%	100%
@Test at column 0	100%	0.8%
closing brace indented	0.4%	99.1%
contains a comment	2.1%	29.3%

The trailing newline alone classifies 8,574 of 8,574 training rows correctly. **All four checkpoints learned the shortcut, and focal loss with a 245× class weight removes the 96.6% non-flaky prior that would otherwise pull the prediction back.**

The probe — same code, different whitespace

Input (n = 40, content held byte-identical)	→ non-flaky
cricket non-flaky, as written	0 / 40
... + trailing newline only	40 / 40
... + indent only	40 / 40
paper's genuinely flaky tests, reformatted	40 / 40

Whitespace decides flaky vs non-flaky. so any flaky/non-flaky number from this model is suspect — including the published 0.99 headline.

Run A's best class was an artifact. order dependency scored 75% recall in A and 0% in B; those 9 "detections" were the same no-evidence fallback that also claimed 541 non-flaky tests.

Tokens decide the category. 12/12 correct on the async tests that say sleep/await/latch; 0/8 on the adversarial tests with the same bug behind day.drinks() — all 8 called non-flaky.

And in reverse. 4 deterministic decoys stuffed with concurrency words: 4/4 called flaky, against a 6.8% base rate.

How we ran it

Six stateful Maven goals that locate what an async-wait test is racing against, then emit a patch inserting a barrier so the assertion cannot run early.

Six goals, in order, one test at a time

```
T='com.cricket.live.LiveScoreIngestTest
  #scorecardReflectsAllBallsAfterIngest'
P=edu.utexas.ece:flakesync-maven-plugin:1.0-SNAPSHOT

mvn $P:concurrentfind -Dflakesync.testName=$T \
    -pl cricket-live      # then the other 5 goals
```

Order matters. each goal reads the previous one's output file, so none can be skipped.

Budget. the paper reports a median of 58.77 minutes per test; delaylocs and deltadebug dominate.

Target selection

Test carrying a flakesync block	Expected
LiveScoreIngestTest#scorecardReflects...	repairable — our target
BroadcastLatencyTest#everyBattingMile...	repairable
MatchTimingTest#theArchivelsWritten...	not repairable
MatchTimingTest#theOverlayKeepsUp...	not repairable

We ran the primary target, **written to mirror Figure 1 of the paper. Ground truth for it: critical point ScorecardUpdater.onBall (final statement), barrier in LiveScoreIngestTest before the assertEquals.**

Results

8 m 18 s

end-to-end, all six goals

53 → 1

locations after delta debugging

Repaired

test passes 50/50 once patched

Goal by goal

Goal	Time	Outcome
concurrentfind	12 s	90 concurrent methods
delaylocs	1 m 46 s	53 locations; fails at delay 100
deltadebug	1 m 52 s	minimised to WorkerPool#59
critsearch	2 m 36 s	root WorkerPool.beforeExecute
barrierpointsearch	1 m 48 s	barrier at #105, threshold 240
patch	4 s	2 patch files

Fast because it never had to search. **delaylocs failed at the first delay value it tried and deltadebug collapsed 53 locations to one, so neither phase had to explore.**

Against ground truth

Expectation	Result
repairable: true	yes
barrier before the assertEquals	yes — line 105
critical point = ScorecardUpdater.onBall	no — WorkerPool

The emitted patch, actually run

	isolated ×10	rerun ×50
before patch	10 pass	0 fail
patch as generated	10 hang	— hangs
threshold corrected	10 pass	0 fail

Observations

A correct barrier reached through a coarser cause

Stage	What it produced
delaylocs	ranked ScorecardUpdater#69 first — exactly the statement ground truth names
deltadebug	minimised to WorkerPool#59, started.incrementAndGet() — the cheapest single choke point
critsearch	explored outward from that minimum, so it never revisited the listener
failure mode	a delay in beforeExecute stalls every task — the run threw an NPE, not the assertion failure

Minimality and explanatory value are not the same objective. **A global choke point will always win a minimisation race against the specific listener whose write the assertion actually reads.**

The patch deadlocks as generated

```
WorkerPool.resetExecutions();      // -> 0
pipeline.submitAll(...);
while (WorkerPool.getExecutedStatus() < 240)
    Thread.yield();                // never true
```

The threshold and the reset disagree. 240 = 120 warm-up tasks + 120 from the test, but the counter is reset after the warm-up has already run, so only 120 can ever be counted.

One integer fixes it. waiting on DELIVERIES makes the count test-local, so the repaired test behaves the same in isolation and in the full suite.

Evaluation warning. "did it emit a patch" scores this a success; running the patched test scores it a hang. One constant separates the two verdicts.

How we ran it

A Java agent that injects a 100 ms delay at every tracked field and API access to force races into the open, and reports the conflicting pairs it observes.

Install the plugin, then run the reduced target

```
cd TSVD4J && mvn clean install
cd ../cricket-tracker
mvn -Ptsvd4j $PLUGIN:tsvd4j -pl cricket-live
    # then cricket-stats, cricket-api
```

Scope. the -Ptsvd4j profile narrows Surefire to 6 classes / 29 tests across 3 modules; the full suite under a 100 ms delay per access would never finish.

Findings land in .tsvd4j/ per module — an aggregate Conflicting-Pairs.txt plus one file per test.

Four things worth knowing first

The short goal prefix does not resolve. mvn tsvd4j:tsvd4j fails — edu.utexas.ece is in no pluginGroup, so fully-qualified coordinates are required.

The profile does not wire the plugin in. it only narrows Surefire's <includes>; the plugin still resolves from ~/.m2.

Conflicting-Pairs.txt is appended at each JVM shutdown. 54 lines for 27 unique pairs — always sort -u.

A missing file means zero findings, not a crash. cricket-stats and cricket-api produced only listener.log.

Results

6 / 6

expected pairs found

0 / 2

control false positives

27

unique conflicting pairs

1.65 M

thread events tracked

7 h 24 m

wall time, 29 tests

The six pairs ground truth names — all reported

Expected field / call	Reported as	Source
ScoreCard.legalBalls	ScoreCard 89	legalBalls++
ScoreCard.totalRuns	ScoreCard 81	totalRuns += runs
Innings.balls add	Innings add 136	balls.add(ball)
MatchRegistry.matches put	MatchRegistry put 35	matches.put(...)
MatchRegistry.registrations	MatchRegistry 36	registrations++
LiveFeedBroadcaster.subscribers	LiveFeed... add 94	subscribers.add(...)

Both designated controls stayed clean. **RequestMetrics (AtomicLong + ConcurrentHashMap) across 1 M events, and Match.dirty — the shape the paper itself names as a false-positive source.**

Where the run spent its time

Module	Time	Tests	Failed	Pairs
cricket-live	40 m	17	15	27
cricket-stats	1 h 46 m	8	2	0
cricket-api	4 h 58 m	4	1	0
Total	7 h 24 m	29	18	27

cricket-live yields 100% of the findings in 9% of the runtime. **Cost tracks tracked accesses, not test count — a 500× spread per test, and cricket-api's 4 tests cost more than the other 25 combined.**

Observations

Where the 27 findings landed

Category in the reduced target	Tests	Pairs
concurrency	7	27
async wait	6	0
unordered collections	4	0
non-flaky controls	6	0
unlabelled	6	0

Category discrimination is the strongest result. **All 27 pairs trace to the 7 concurrency tests, and none to the async, unordered-collection or control tests — despite 13 of those failing during the run. It does not merely find races; it declines to attribute non-race flakiness to races.**

Observations

It beat rerunning on one test. theBatterIsCreditedWithEveryRun fails 0/50 in isolation; under delay injection it failed and produced 23 of the 27 pairs.

Overhead breaks the controls. 5 of 6 non-flaky controls failed, every one on an awaitIdle timeout rather than a substantive assertion. Artifacts, not regressions.

Failure counts carry no rate signal. a 0/50 test and a 43/50 test both report expected 120 but was 4 — these must not be fed back into the labels file.

Score the aggregate, not the per-test files. two tests got no per-test file at all; per-test recall would read 4/6 instead of the true 6/6.

One unrecorded finding. ScoringRules|30:92 is not in ground truth and may warrant a labels update.

Three artifacts, three different verdicts

Artifact	Ran	Against ground truth	The observation that matters
FlakyLens	19 min / run	0.03 or 0.88, depending on whitespace	Decides flaky vs non-flaky on source formatting, not on code. Tokens, not behaviour, pick the category.
FlakeSync	8 m 18 s	repaired; barrier exact, critical point coarser	The emitted patch deadlocked. Review before applying — one integer separated repair from hang.
TSVD4J	7 h 24 m	6/6 pairs, 0 false positives	Declines to attribute non-race flakiness to races — even when the test is failing in front of it.

Cross-cutting

The two dynamic artifacts held up; the learned one did not. TSVD4J and FlakeSync both found what the labels file said they should. FlakyLens' headline number is one byte from collapsing.

Instrumentation overhead is not a result. 5 controls failed under TSVD4J on timeout guards alone — discount those, and score the aggregate output, not the per-test files.

Every artifact needed a port. prefix resolution, CRLF line endings, hardcoded CUDA — none of it in the papers, all of it in the runbooks.